// Power Platform & Fabric User Group Cologne · 8 Dec 2026

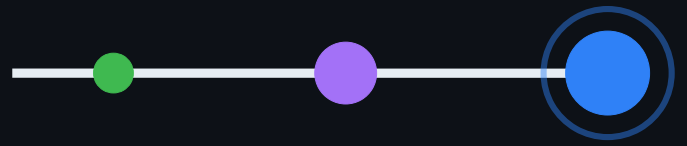
Stop Copy-Pasting Your Pipelines

Metadata-driven orchestration in practice



David Fengler

Azure Cloud Architect · glueckkanja AG





// whoami

David Fengler

Azure Cloud Architect @ glueckkanja AG

I build data foundations — lakehouses, pipelines, governance — and turn them into AI that holds up in production, not just in demos.

- blog datadave.dev
- github github.com/phant0mw0lf
- linkedin linkedin.com/in/davidfengler
- oss pipetree.dev

// agenda

What we'll cover

01 **The problem** ten systems, ten copies

02 **The model** YAML schema and lakehouse layout

03 **The executor** parallel runs, retries, failures, logging

04 **Two ways on Fabric** the package vs a pipeline with an activity per table

05 **Design decisions** what worked, what didn't

06 **Live demo** a new source in Fabric, end to end

// 01

Ten systems, ten copies

How a data platform grows when copying is the fastest way forward



// 01 · the problem

It started with one good notebook

01

ERP first

Dynamics 365 via Export to Data Lake. Parameterised from day one — new instances never touched the file.

02

Then the rest

CRM via Synapse Link, SharePoint and Entra via Graph, on-prem SQL Server over JDBC. Copy, swap the reader, adjust.

03

Ten files

Each one generic within its system. Each one its own step in the job. The problem sat between the files.

// 01 · the problem

Then came the cross-cutting change

10

source systems, each with its own copied file

1

change: lineage columns everywhere, then multithreading

10×

the same edit, ten times, drifting a little each time

// 01 · the insight

What differed between the systems wasn’t logic. It was description.

which system · which object · which key · how to merge

// 02

Describe it, don’t write it

One YAML file, one small package that interprets it



// 02 · the model

Four things, kept apart

01

What

The metadata: tables, systems, keys, merge strategy.

02

In what order

A dependency graph the package derives itself.

03

How

An engine-agnostic executor. Spark is an adapter.

04

Where

Versioned files in the repo, promoted through CI/CD.

// 02 · the model

One table, one block

pipeline.yaml

```yaml
systems:                        # declared once, referenced by name
  crm_dataverse:
    type: csv                    # csv · json · parquet · sqlserver · kusto · custom …
    path: data/customer.csv

bronze:
  tables:
    customer:
      source: { system: crm_dataverse, object: account }
      business_key: [accountid]
      strategy: scd2                # scd2 · scd1 · replace · append
```

// 02 · the model

Merge is the strategy, full stop

strategy

- scd2 — full history
- scd1 — latest state
- replace — rebuild every run
- append — add rows

merge knobs

- sequence_by — which change wins
- delete_when — what marks a delete
- delete_mode — soft · hard · ignore
- ignore_columns — not a real change

// 02 · the model

Dependencies come from the code

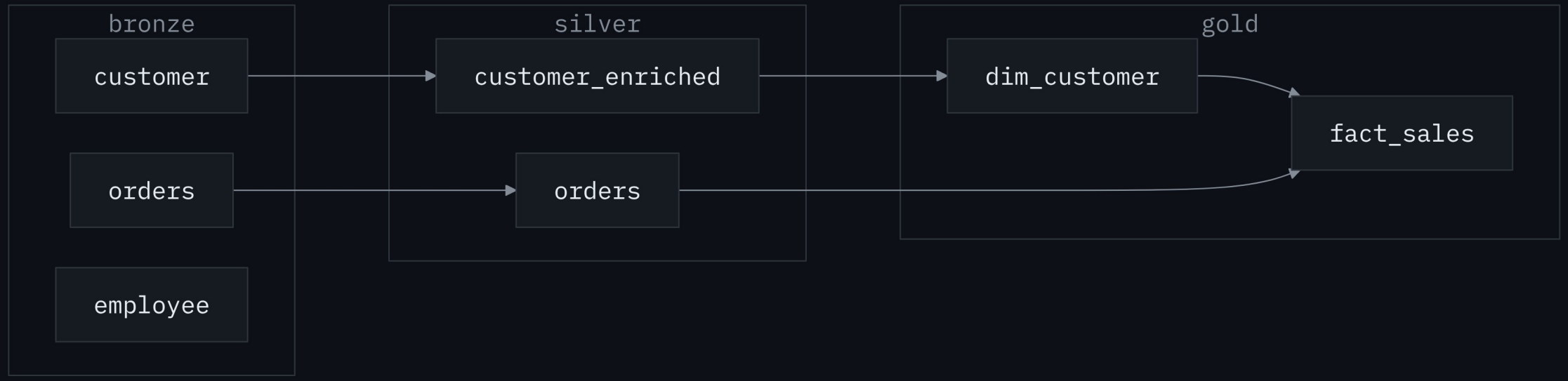
notebooks/gold/fact_sales.py

```
# one output table per file, inputs read at the top
orders   = spark.table("silver.orders")
customer = spark.table("gold.dim_customer")
# everything after — joins, logic — is yours

# pipeline.yaml
fact_sales:
  logic: notebooks/gold/fact_sales.py
  depends_on: auto        # parsed from the reads above
```

// 02 · the model

The package draws the tree



topological order · cycle check · run a table plus everything downstream

// 02 · the lakehouse

One lakehouse, a schema per layer

01

A schema per layer

All layers in one Fabric lakehouse, not one lakehouse per layer. The example uses bronze · silver · gold.

02

Files/

pipeline.yaml, the logic notebooks and landed source files. Versioned in the repo, deployed together.

03

_meta

pipeline_run_log: one row per table and run. Audit columns on every row, keyed to the same run.

Layers are names, not stages

one task per layer

- Every new layer is a new job step and new wiring
- The slowest table in a layer holds up the next layer
- So everything gets squeezed into three layers

when the tree drives execution

- A layer is a top-level key in the YAML — name it anything
- landing · staging · intermediate · serving, or ten of them
- Same table name in every layer: staging.orders, serving.orders
- Execution follows the tree, so a layer costs nothing

// 02 · the lakehouse

Audit columns you never write again

every table

- _inserted_at · _updated_at
- _is_deleted
- _execution_id
- _source_system

scd2 tables, additionally

- _valid_from · _valid_to
- _is_current
- timestamps: UTC bigint, YYMMddHHmmssSSS
- readable in a debug query, lines up with the run

// 02 · alternatives

You don’t need my package

01

Fabric / Data Factory

Metadata-driven copy on control tables. Strong at ingestion — and stops there.

02

dbt

Transformations as configuration, in SQL. Never touches the extract.

03

Databricks dlt-meta

Metadata-generated Lakeflow pipelines. Databricks only.

04

pipetree

One YAML from source to star schema, the same on Fabric and Databricks.

// 03

Run the tree, not the layers

Parallel, with retries, and failures that stop only what depends on them



// 03 · the executor

One counter per table

executor (trimmed)

```
pending = {t: len(parents[t]) for t in run_set}
submit(t for t, n in pending.items() if n == 0)

while futures:
    done = wait(futures, return_when=FIRST_COMPLETED)
    for table in done:
        for child in children[table]:
            pending[child] -= 1
            if pending[child] == 0: submit(child)
```

// 03 · the executor

Retry what can be retried

transient → retry

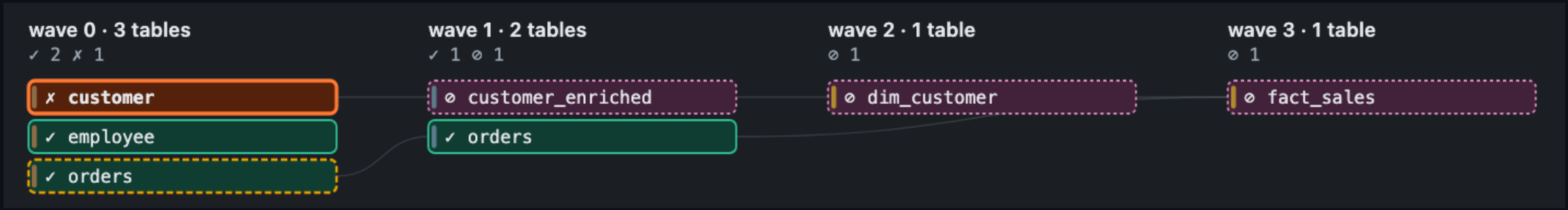
- Throttling, timeouts, dropped connections
- Spark/Delta errors translated by the adapter (0.1.1)
- Up to 3 attempts, full-jitter backoff

permanent → fail now

- Malformed export, missing column
- Waiting never fixes these
- append retries only if the run’s rows can be deleted first

// 03 · the executor

A failure stops exactly its subtree



failed ≠ upstream_failed — one error to read, not seven

// 03 · the executor

The digest, and an honest exit code

run output

```
bronze.orders: attempt 1 failed (Throttled: 429), retrying in 0.47s
bronze.orders: retried→succeeded (attempts=3)

Run 261015095124804: FAILED (exit code 1)
  failed              bronze.customer
  succeeded           bronze.employee
  retried→succeeded   bronze.orders
  upstream_failed     silver.customer_enriched
  upstream_failed     gold.dim_customer
  upstream_failed     gold.fact_sales
  succeeded           silver.orders
```

// 03 · logging

Every run leaves a trail

01

Run log

_meta.pipeline_run_log: status, attempts, duration, error per table. Written once per run, idempotent on _execution_id.

02

Live graph

LiveGraphView redraws the tree in the notebook while tables run. HtmlFileObserver does it anywhere else.

03

Alerts

A failed run can send one email via Azure Communication Services — token credential, no connection string.

// 04

Two ways on Fabric

The same seven tables — once in one activity, once as an activity per table



// 04 · two ways on fabric

Same notebook, different orchestrator

01

A — one activity

A data pipeline with a single Notebook activity. pipetree owns the tree, the retries and the run log.

02

B — activity per table

One Notebook activity per table, dependsOn generated from the YAML. The pipeline owns the tree.

// 04 · two ways on fabric

Seven tables, clean run

53 s

A: pipetree for all seven tables, inside one activity

924 s

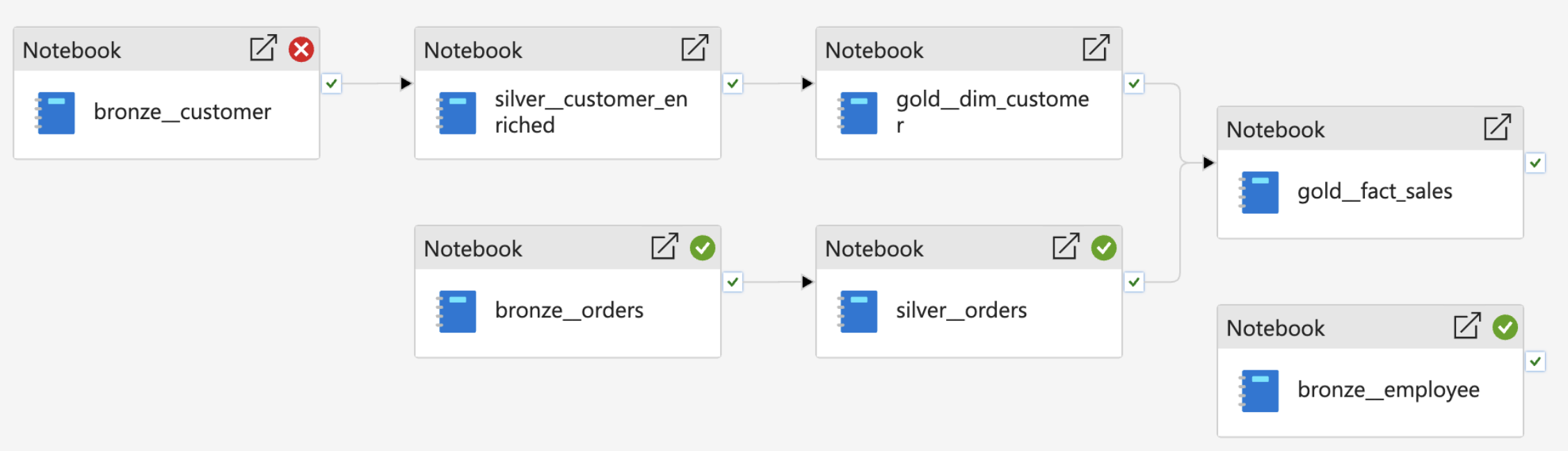
B: every activity is its own session — four session starts on the critical path

120

activities max per Fabric data pipeline — 171 tables need two

// 04 · two ways on fabric

B: the pipeline's view of a failure



dependents stop — but the activity retry ran a malformed file three times

// 04 · two ways on fabric

Which file do you rewrite when you leave?

A — the package

- Nothing: pipeline.yaml and the logic files
- Same bytes on a laptop, Fabric and Databricks
- The notebook is a few lines of seam
- Activity retry: set it to zero, the package retries

B — a pipeline per table

- pipeline-content.json, every line of it
- The tree restated in a format only Fabric reads
- Monitor knows activities, not tables
- Stops at 120 activities per pipeline

Not reach — what reach costs

data pipeline · gateways

- On-premises data gateway: a Windows service next to the SQL Server
- VNet data gateway for sources in an Azure VNet
- Neither is available to a notebook
- Hybrid: Copy into a landing folder, storage_stream from there

notebook · managed VNet

- Managed private endpoints to Azure SQL, storage, Private Link — on-prem SQL Server included
- Then sqlserver is a plain JDBC read, custom covers the rest
- No starter pools: 3–5 min per session start

// 05

What worked, what didn’t

And what I’d change today



// 05 · design decisions

Looking back

01

Worked

depends_on: auto. Tree over layers. failed vs upstream_failed. No locks in the scheduler.

02

Hurt

The simple parser misses dynamic table names. Engine errors needed translating before retries worked. append is hard to retry.

03

Today I'd …

Stop selling bronze · silver · gold as the rule. It is one example — use the layers the data needs.

// 06

A new source, live

products.parquet → bronze → gold.fact_sales → Power BI



$ live demo_

%pip install pipetree-meta · Fabric notebook · Direct Lake report

// 06 · recap

Everything the new source took

pipeline.yaml (+ one join in fact_sales.py)

```yaml
systems:
  shop_products:
    type: parquet
    path: data/products.parquet

bronze:
  tables:
    products:
      source: { system: shop_products, object: products }
      business_key: [product_id]
      strategy: scd1
```

// 06 · recap

One new node, one new edge



The same change in variant B

A — the package

- One system block, one bronze table
- One join in fact_sales.py
- The tree picks up the new edge

B — a pipeline per table

- The same YAML and the same join
- Plus a new activity with the right dependsOn
- Or re-run the generator — the YAML is the source anyway
- And one step closer to 120

// takeaway

A new source is a config change, not a copy.

describe it once · let the package run it

// try it

Take it home

01

Install it

pip install pipetree-meta — docs and quick start at pipetree.dev. In Fabric: one %pip cell in the notebook, an environment for scheduled runs.

02

The long version

The “Pipeline Engineering” series on datadave.dev. Part 3 is the Fabric comparison, with code in datadave-fabric-examples.

03

Slides & demo

PDF, pipeline.yaml, sample data and the notebook on the talk page.

// thanks

Thank you.

Questions, war stories, pushback — all welcome.

- blog datadave.dev
- github github.com/phant0mw0lf
- linkedin linkedin.com/in/davidfengler
- oss pipetree.dev



datadave.dev/talks